N1

Дан массив весов $a_1, a_2, \ldots, a_n$. Корабль за одну переправу может перевозить не более t.

Определение динамического программирования

Определим $dp[i][j]$ как минимальное число переправ для перевозки объектов с индексами от i до j.

Переход

$$dp[i][j] = 1 + \min_{i \le k \le j, \sum_{m=i}^{k} a_m \le t} dp[k+1][j], \quad \min_{i \le l \le j, \sum_{m=l}^{j} a_m \le t} dp[i][l-1]$$

Где: - $dp[k+1][j]$ — если мы перевезли префикс. - $dp[i][l-1]$ — если мы перевезли суффикс.

Базовые случаи

$$dp[i][j] = 0, \quad \text{если } i > j.$$

Реализация за $O(n^2)$

Мы заполняем таблицу dp в порядке увеличения разницы $j - i$.

```
for len = 0 to n-1:
    for i = 1 to n-len:
        j = i + len
        вычисляем dp[i][j]
```

Ответом будет $dp[1][n]$.

N2

Рассмотрим задачу о рюкзаке: даны n предметов с весами $w_1, w_2, \ldots, w_n$ и стоимостями $v_1, v_2, \ldots, v_n$, а также рюкзак вместимости W. Требуется выбрать подмножество предметов, чтобы суммарный вес не превышал W, а суммарная стоимость была максимальной.

Стандартное динамическое программирование

Определим массив $dp[w]$, где $dp[w]$ — максимальная стоимость, достижимая при точной вместимости w. Он обновляется по формуле:

$$dp[w] = \max(dp[w], dp[w - w_i] + v_i).$$

Этот метод требует $O(nW)$ времени и $O(W)$ памяти.

Оптимизация памяти

Чтобы достичь $O(n + W\sqrt{n})$ памяти:

- Разделим предметы на группы по $B = \lceil \sqrt{n} \rceil$ элементов.
- Будем хранить массивы dp и $prev$ размера $O(W\sqrt{n})$.
- Запоминаем переходы для восстановления ответа.

Алгоритм

1. Разбиваем предметы на блоки по $B = \lceil \sqrt{n} \rceil$.
2. Для каждого блока обновляем dp только на этом подмножестве предметов.
3. Запоминаем переходы в массиве $prev$.
4. Восстанавливаем ответ, двигаясь по $prev$ от W до 0.

Сложность

- Временная сложность: $O(nW)$.
- Пространственная сложность: $O(n + W\sqrt{n})$.

N3

Дано n неотрицательных целых чисел $a_1, a_2, \ldots, a_n$ с суммой m. Требуется минимизировать:

$$f(a) = \max_{1 \le i,j \le n} \frac{a_i}{a_j}$$

Оптимальная стратегия

Лучший способ минимизировать $f(a)$ — сделать элементы как можно более равными. Пусть:

$$k = \lfloor m/n \rfloor, \quad r = m \mod n$$

Тогда:

- r элементов принимают значение $k + 1$.
- Остальные $n - r$ элементов принимают значение k.

Тогда:

$$f(a) = \frac{k+1}{k}$$

Решение за $O(m^2 n)$

- Перебираем все возможные разбиения суммы m.
- Для каждого разбиения вычисляем $f(a)$.
- Находим минимум среди всех возможных разбиений.

Сложность: $O(m^2 n)$.

N4

- Используем бинарный поиск по k.
- Определяем, сколько чисел должно быть $k + 1$.
- Вычисляем $f(a)$ за $O(\log n)$.

Сложность: $O(m^2 \log n)$.

Решение задачи о сумме по подмножествам

Дан массив $a(0), \ldots, a(2^n - 1)$. Определим новую функцию:

$$a'(mask) = \sum_{\text{submask} \subseteq \text{mask}} a(\text{submask}).$$

Приведённый ниже алгоритм вычисляет a' на месте:

Algorithm 1 Обновление массива по подмножествам

```
for i = 0 to n − 1 do
    for mask = 0 to 2^n − 1 do
        if бит i в mask равен 0 then
            a[mask + (1 ≪ i)] ← a[mask + (1 ≪ i)] + a[mask]
        end if
    end for
end for
```

Рассмотрим вычисление суммы по подмножествам индукцией:

- **База:** при $i = 0$ маски различаются только в первом бите, и сумма корректно передаётся.
- **Переход:** на каждом шаге алгоритм обновляет суммы для всех масок, добавляя вклад новых подмножеств.

После завершения алгоритма:

$$a'(mask) = \sum_{\text{submask} \subseteq \text{mask}} a(submask).$$

Асимптотика

Так как мы проходим по всем 2^n маскам и обновляем каждую $O(n)$ раз, сложность алгоритма:

$$O(n \cdot 2^n).$$

N5

Формулировка задачи

Даны n предметов с весами $w_1, w_2, \ldots, w_n$ и рюкзак вместимости W. Нужно найти максимальный суммарный вес, который можно поместить в рюкзак.

Идея решения

Используем **битовый массив** dp, где $dp[i] = 1$ означает, что вес i достижим.

1. Заводим битовый массив длины $W + 1$, ставим $dp[0] = 1$.
2. Для каждого предмета выполняем:
$$dp = dp \mid (dp \ll w_i)$$
3. Ответ — максимальное i, для которого $dp[i] = 1$.

Алгоритм

Algorithm 2 Решение задачи с битсетами

$dp \leftarrow$ битовый массив длины $W + 1$
$dp[0] \leftarrow 1$
for $i = 1$ **to** n **do**
    $dp = dp \mid (dp \ll w_i)$
end for
Вывести $\max\{i \mid dp[i] = 1\}$

Анализ сложности

- Используем битовый массив длины $W + 1$, где каждая операция работает за $O(W/w)$.
- Всего n итераций, значит общая сложность:
$$O(nW/w).$$

N6

Формулировка задачи

Дан набор строк $s_1, s_2, \ldots, s_n$ из маленьких латинских букв с суммарной длиной:

$$S = \sum_{i=1}^{n} |s_i|$$

Каждая строка s_i имеет стоимость a_i. Определим стоимость произвольной строки t длины l как:

$$\sum_{i=1}^{n} \mathrm{cnt}(s_i, t) \cdot a_i$$

где $\mathrm{cnt}(s_i, t)$ — число вхождений s_i в t. Требуется найти строку длины l с максимальной стоимостью.

Решение через Ахо-Корасик и матричное ускорение

1. Строим автомат Ахо-Корасика для множества строк $s_1, \ldots, s_n$.
2. Определяем матрицу переходов T, где T_{ij} — переход между вершинами бора.
3. Используем динамическое программирование:
$$dp_k = T^k \cdot dp_0$$
4. Находим dp_l с помощью бинарного быстрого возведения в степень за $O(S^3 \log l)$.

Асимптотика

- Построение автомата: $O(S)$
- Размер матрицы переходов: $O(S \times S)$
- Быстрое возведение в степень: $O(S^3 \log l)$

Таким образом, итоговая сложность:

$$O(S^3 \log l).$$

N7

Формулировка

Дано множество чисел $a_1, a_2, \ldots, a_n$ с суммой:

$$N = \sum_{i=1}^{n} a_i.$$

Требуется для каждого $x \in [0, N]$ определить, можно ли составить сумму x.

Решение через битсет

Используем динамическое программирование с битовыми сдвигами:

- Введём битовый массив dp длины $N+1$, где $dp[x] = 1$, если сумма x достижима.
- Разделим числа на малые ($a_i \leq \sqrt{N}$) и большие ($a_i > \sqrt{N}$).

Алгоритм

Algorithm 3 Нахождение достижимых сумм

$dp[0] \leftarrow 1$
for каждое $a_i \leq \sqrt{N}$ **do**
    $dp = dp \mid (dp \ll a_i)$
end for
for каждое $a_i > \sqrt{N}$ **do**
    for $x \leftarrow N$ **downto** a_i **do**
        $dp[x] \leftarrow dp[x] \vee dp[x - a_i]$
    end for
end for

Асимптотика

- Битовый сдвиг для малых чисел: $O(N)$.
- Обход больших чисел (не более $O(\sqrt{N})$): $O(N\sqrt{N})$.
- Итоговая сложность:

$$O(N\sqrt{N}).$$